Algoritmi paraleli si distribuiți

7. Algoritmi de sortare în paralel

+ CUDA

Sortarea în paralel

- Utilitatea
  - Sortarea este în general o operație des folosită în aplicații.
  - Implicație: Permite căutare binară (rapidă!) in O(log n)
- Scopul
  - Obținerea accelerării la sortarea unui vector de valori folosind *n* procesoare.
- Accelerarea
  - Cei mai buni algoritmi secvențiali au complexitate *O(n log n)*
  - Cea mai mare accelerare posibilă a unui algoritm paralel care utilizează n procesoare este O(log n) = O(n log n)/n

Algoritmi de sortare

- Task-ul este să aranjăm în ordine o colecție neordonată
  - Nu este neaparat mulțime: putem avea duplicate
  - Elementele nu sunt neaparat numere, putem avea puncte în plan sau alte obiecte
- Raspunsul este practic o permutare a elementelor
- Metode bazate pe comparații
  - Se compară perechi de elemente și se schimbă între ele
  - *O(n log n)*
- Metode care NU se bazează pe comparații
  - Bucket sort, counting sort, radix sort

Sortarea în paralel

- Unde se află elementele care se sortează?
  - Trebuie să fie distribuite între toate procesele.
  - Ordinea obținută nu trebuie să depindă de ordinea de execuție a proceselor.
- Cum sunt efectuate comparațiile?
  - Dacă avem un element per proces atunci comunicarea între procese va fi semnificativă (va domina) timpul de execuție: *compară-schimbă*.
  - Dacă avem mai multe elemente per proces atunci va trebui efectuata operația împarte (en., *split*): *compară-împarte*.
- Vine ca abordare alternativă (i.e., distinctă și contrastantă) cu algoritmii de sortare care folosesc memoria partajată.

Proiectarea algoritmilor de sortare în paralel

(V1) Bazată pe un algoritm de sortare secvențial existent

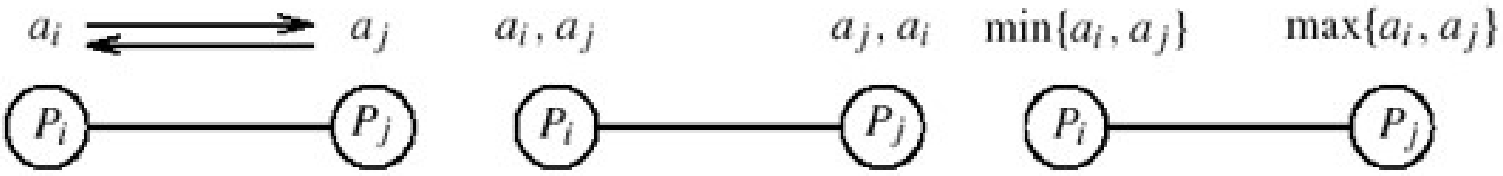
- Încearcă să utilizeze toate resursele disponibile.
- Poate transforma un algoritm secvențial slab într-un algoritm paralel rezonabil de rapid
  - Ex. Bubble sort -> Bubble sort parallel (i.e., *odd-even*)

(V2) Abordare complet nouă

- Algoritm nou proiectat de la zero
- Dificil de dezvoltat
- Uneori obține performanțe mai bune (i.e., numai pentru o anumită problemă cu constrângeri specifice referitoare la input, distribuții, data types, etc.).

Sortarea în paralel (cont.)

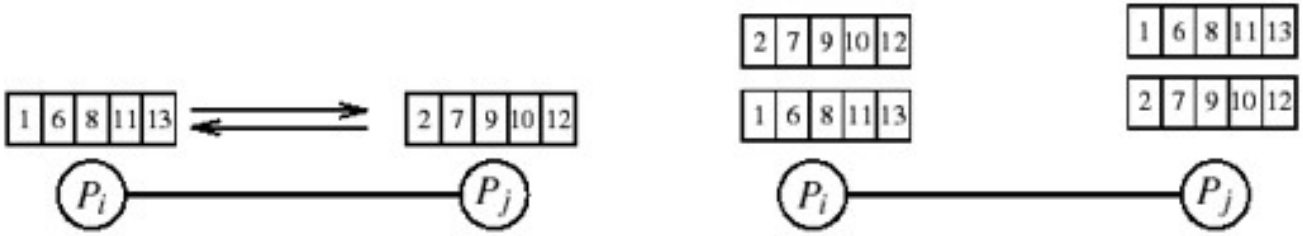
- Un element per proces: *compară-schimbă*



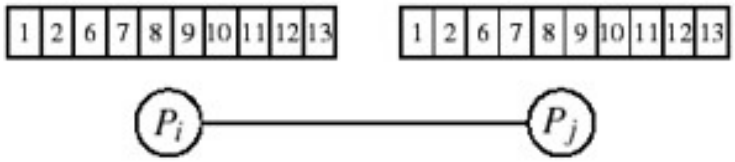
Pas 1. Schimbă Pas 2. Compară

Sortarea în paralel (cont.)

- Mai multe elemente per proces: *compară-împarte*



Pas 1. Schimbă



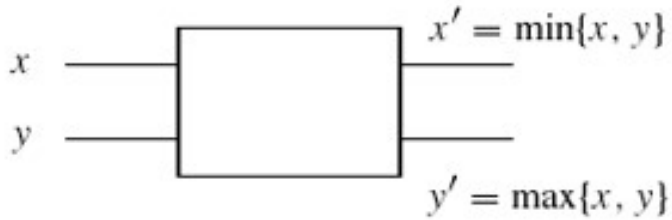
Pas 2. Compară

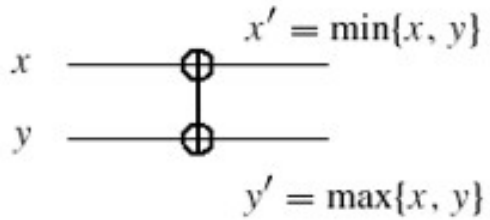
Pas 3. Împarte

Rețele de sortare

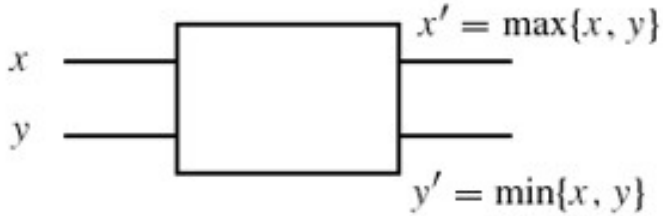
- Rețele care sortează *n* elemente în mai puțin de *O(n log n)*
- Elementul/nodul principal al rețelei este comparatorul
- Comparatoarele sunt conectate în paralel și permută elemente

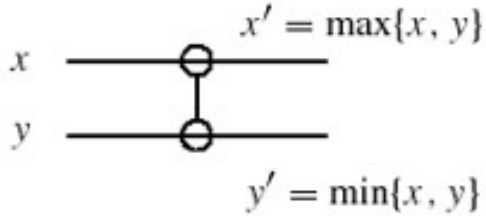
Comparator crescător





Comparator descrescător



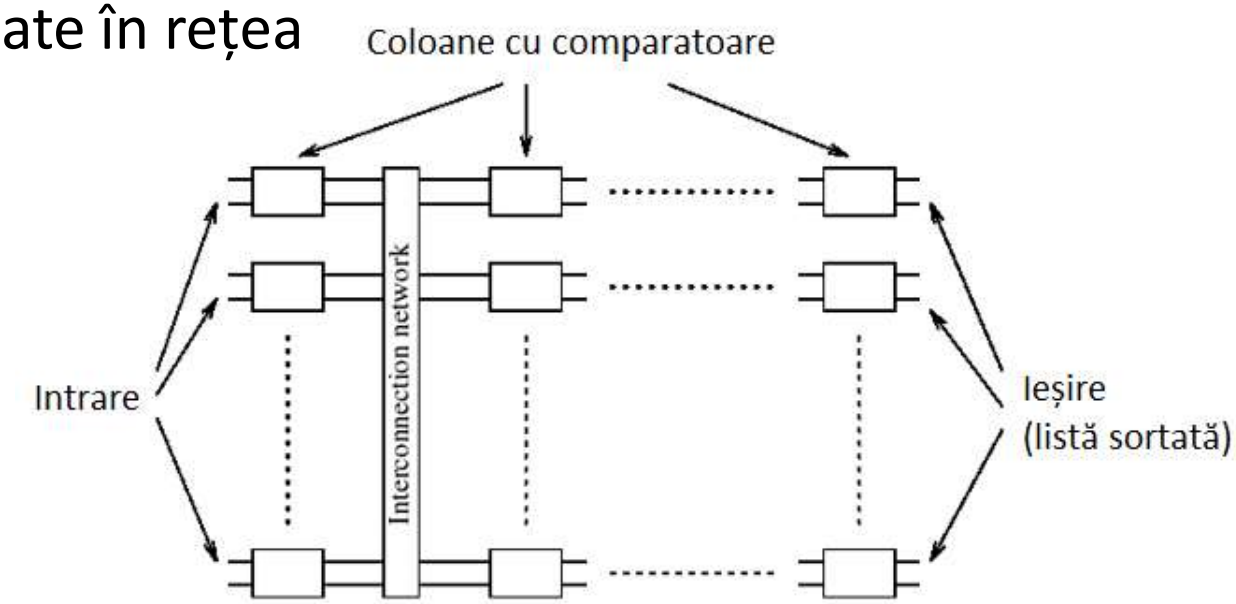


Proiectarea rețelei de sortare

- Sortarea se realizează în etape (i.e., pe nivele)
- Comparatoarele sunt conectate în rețea
- Raspunsul ultimului nivel

este lista sortată

- Timpul este *O(log 2n)*



Sortarea bitonică (en., *bitonic sort*)

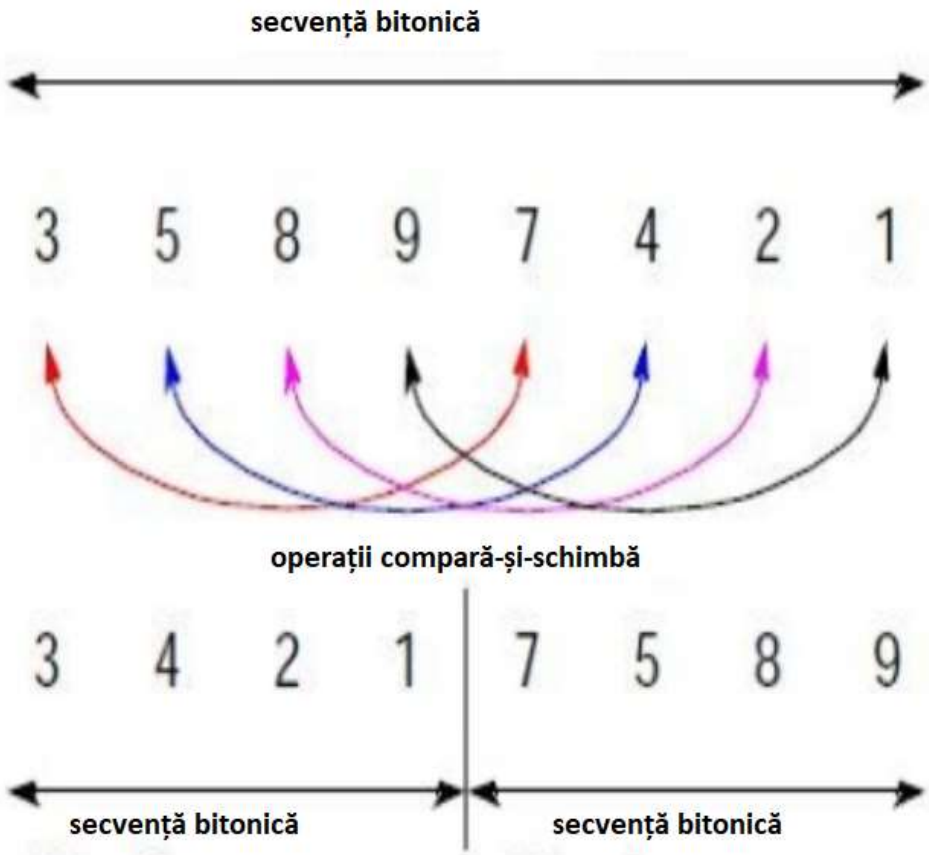
- Secvență bitonică
  - Are elementele $\{a_0, a_1, \ldots, a_{n-1}\}$ unde
  - $\{a_0, a_1, \ldots, a_i\}$ sunt în ordine crescătoare
  - $\{a_i, a_{i+1}, \ldots, a_{n-1}\}$ sunt în ordine descrescătoare

- Secvența bitonică trebuie obținută dintr-o secvență neordonată
  - Se pornește de la o secvență de dimensiune 2 care este implicit bitonică

- Din secvența bitonică se obține apoi lista sortată

Sortarea bitonică: variantă secvențială

- **Problema**: Avem o secvență bitonică și dorim să o ordonăm.
- **Abordare**: divide and conquer
  - Operația *compara-și-schimbă* mută valorile mai mici în stânga, iar valorile mai mari în dreapta.
  - Această operație se aplică recursiv până când secvența devine ordonată.



Sortarea bitonică: variantă secvențială (cont.)

n = lungimea secvenței bitonice

log n = numărul de faze necesare

T(n) = log(n) + T(n/2)

T(n) = log(n) + log(n)-1 + log(n)-2 + ... +1 = log(n) * (log(n) + 1) /2

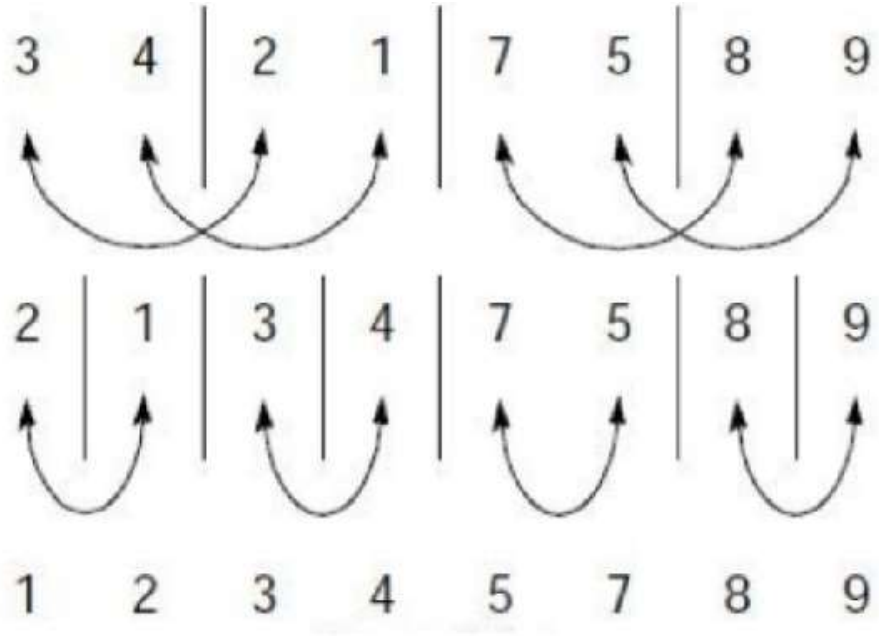
Fiecare fază efectuează n/2 comparații.

În total sunt $O(n*\log^2(n))$ comparații.

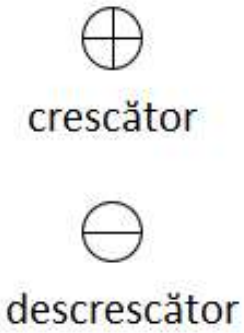
Pentru P=n (n procesoare),

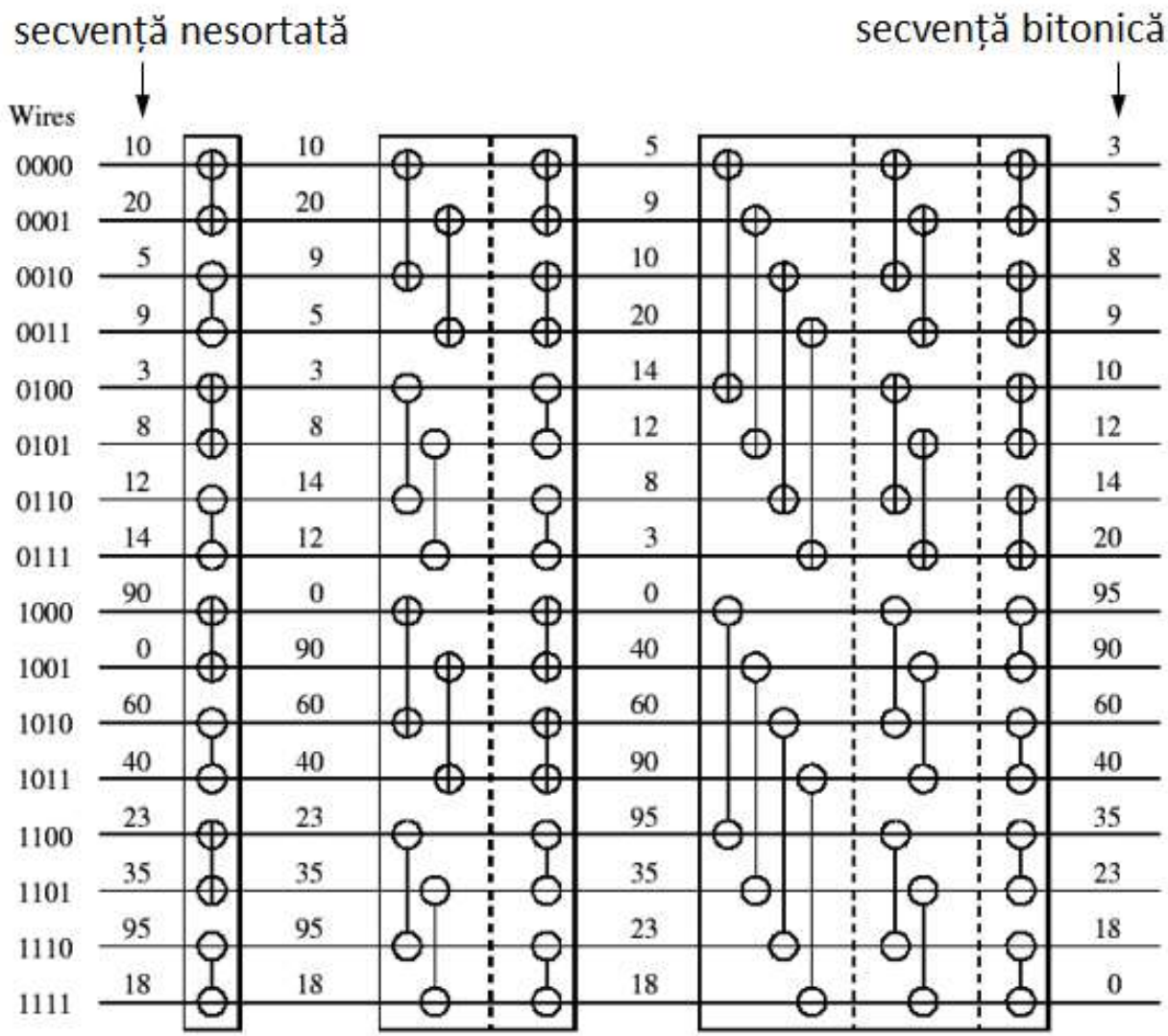
$T(n)= O(n*\log^2(n)) /n =$ **$O(\log^2(n))$**

Ultimele două faze ale algoritmului necesare pentru obținerea secvenței sortate.

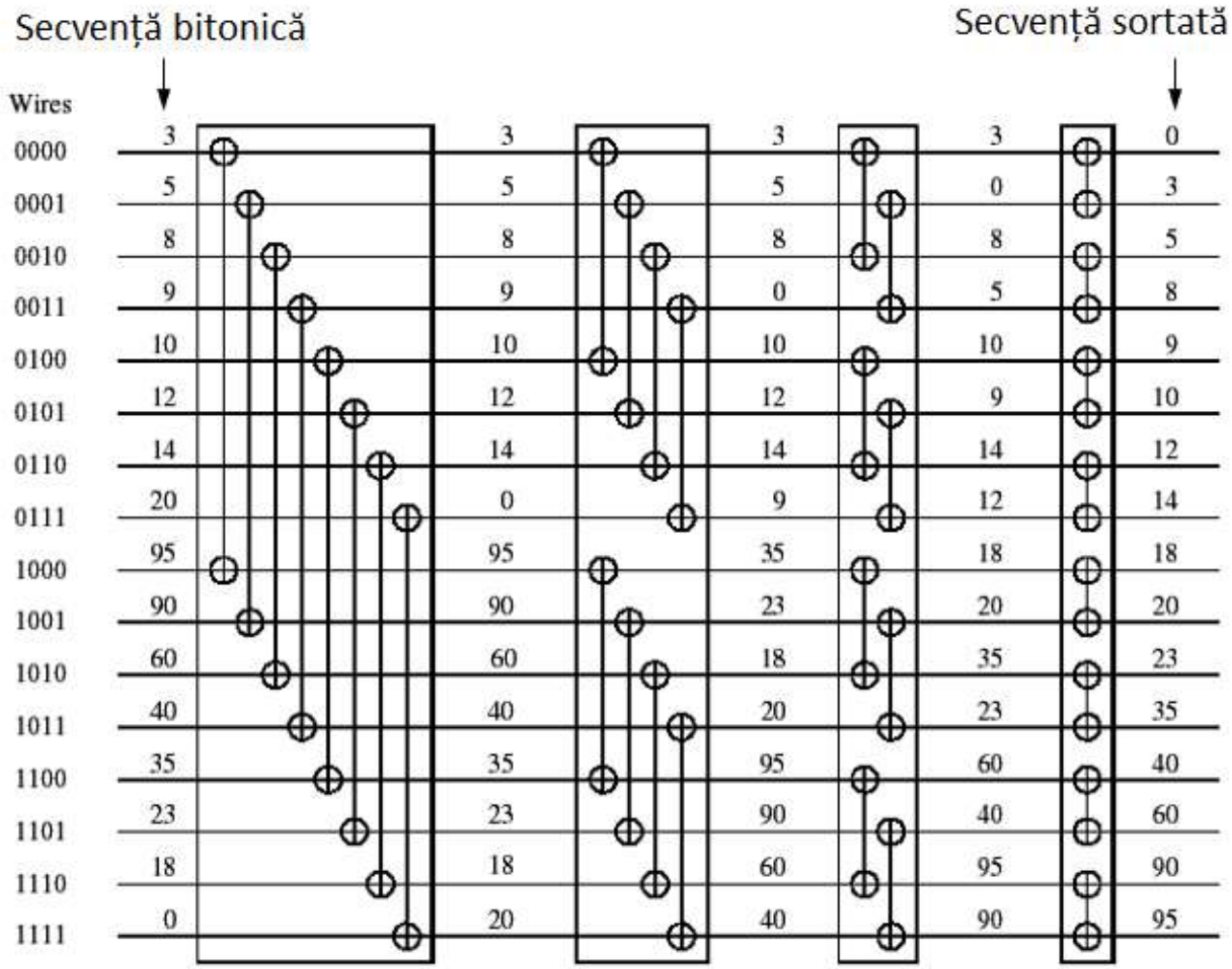


Sortarea bitonică: 1. crearea secvenței bitonice [conectarea procesoarelor]



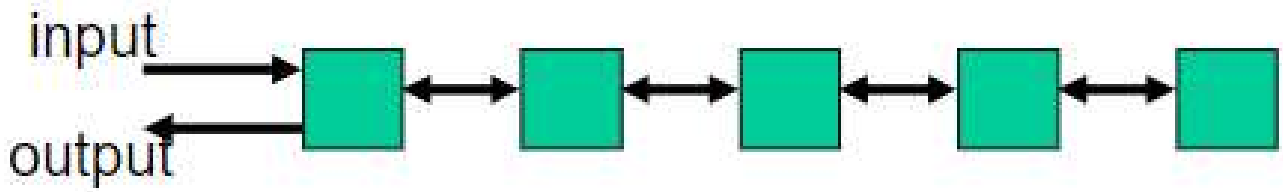


Sortarea bitonică: 2. crearea secvenței sortate [conectarea procesoarelor]



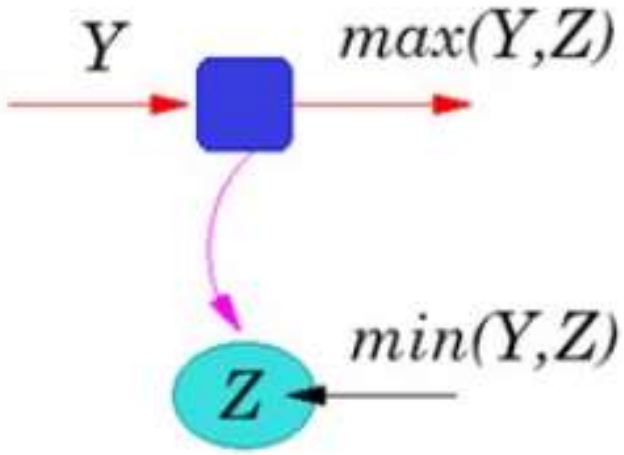
Sortarea cu un array de procesoare

- Fiecare procesor este conectat la ambii vecini (conexiunile sunt bidirecționale, ca într-o lista dublu înlănțuită)
- Toate procesoarele partajează același ceas (sincronism)
- La fiecare tact, procesoarele primesc datele de intrare de la vecini, efectuează calculul, transmit răspunsul către vecini și updatează memoria locală.

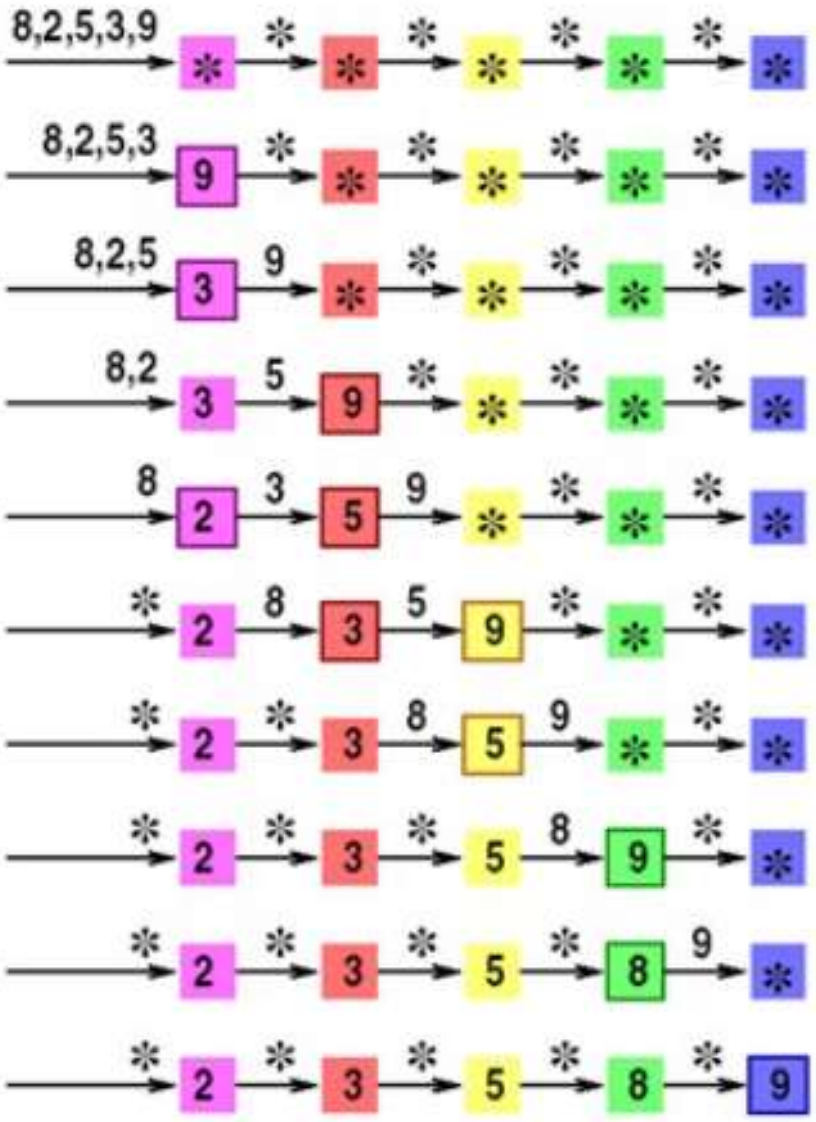


Sortarea cu un array de procesoare

- Logica fiecărui procesor
  - Fiecare procesor stochează valoarea cea mai mică pe care a *văzut-o*
  - Valoarea inițială implicită este ”*”, care este prin convenție interpretată ca fiind mai mare decât orice valoare posibilă din input (i.e., interpretarea poate fi asimilată cu lipsa de semnal sau *nimic*)
  - Cand procesorul primește valoarea Y de la vecinul din stânga, acesta păstrează valoarea mai mică dintre Y si valoarea curentă Z, și transmite valoarea mai mare catre vecinul drept.



Exemplu: Sortarea cu un array de procesoare



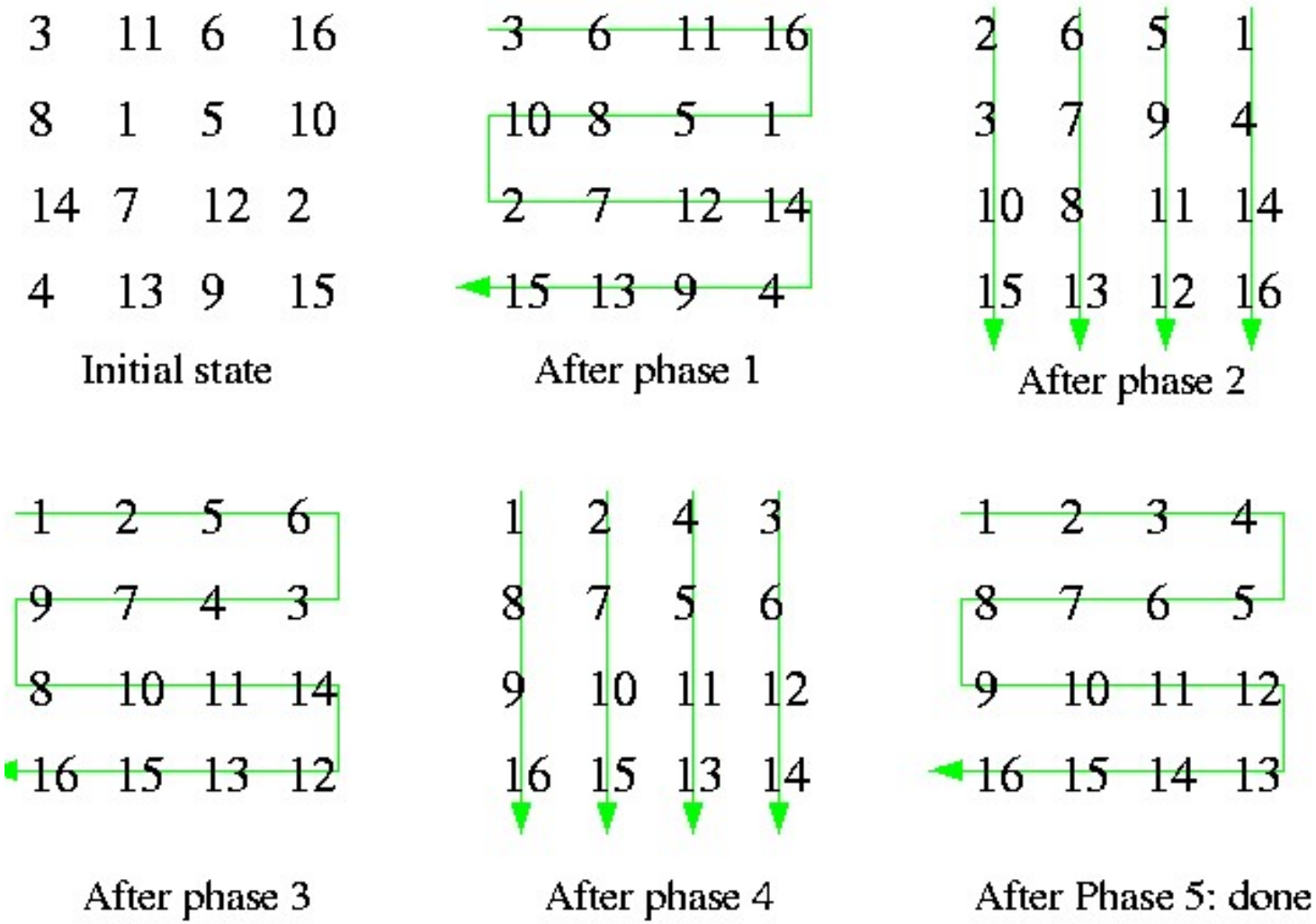
Shear sort

- **Input**: o matrice *n x m*, nesortată
- **Output**: matricea *n x m* sortată șerpuit

Pașii algoritmului

- For i = 1, ... 2 log n + 1
  - If (i este impar) : Faza *rânduri*: Sortăm fiecare rând folosind sortarea odd-even
    - Rândurile impare sunt sortate de la stânga la dreapta.
    - Rândurile pare sunt sortate de la dreapta la stânga.
  - If (i este par): *Faza coloane*: Sortăm fiecare coloană folosind sortarea odd-even

Shear sort: Exemplu



Shear sort

Lemă:

ShearSort trece de *log n+1* ori prin toate rândurile (*if*-ul impar) și de *log n* ori prin toate coloanele (*if*-ul par) pentru a sorta *n*m* valori ale unei matrici *M(n, m)* în ordine șerpuită.

Demonstrație:

- cu ajutorul lemei de sortare 0-1.

Lema de sortare 0-1

Fie M(n,m) o matrice numai cu valori de 0 sau 1.

Există 3 tipuri de rânduri: numai cu 0, numai cu 1, și *mixed* (și 0 și 1).

Obs: Inițial, matricea M poate conține (în cazul cel mai defavorabil) *n* rânduri *mixed*.

Matricea răspuns (sortată șerpuit) conține cel mult un rând *mixed*.

Propoziție:

După fiecare etapă (iterație din *for* = o fază rânduri și o fază coloane) numărul de rânduri *mixed* se reduce cu jumătate.

Lema de sortare 0-1

Lemă: Dacă un algoritm de tipul compară-schimbă reușește să sorteze secvențe de elemente care au numai valorise 0 sau 1, atunci acest algoritm va reuși să sorteze și secvențe de elemente de valori oarecare.

Demonstrație:

Fie algoritmul A și un input de dimensiune N.

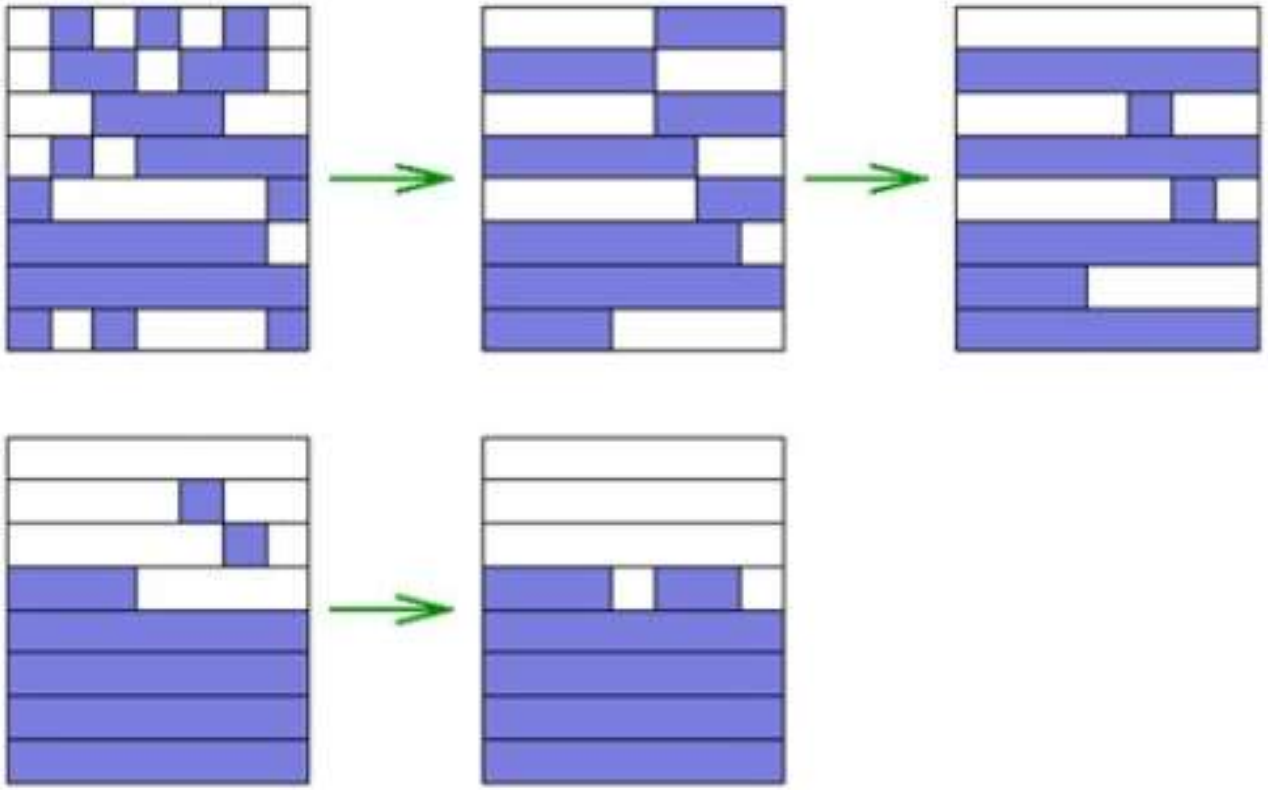
Presupunem că A este corect pe orice input cu valorile 0 și 1.

Presupunem că A nu rezolvă input-ul [x1, x2, …, xN] cu răspunsul [y1, y2, …, yN]. Asta înseamnă că $y_1 \leq y_2 \leq \ldots \leq y_m > y_{m+1}$ pentru un anumit m.

Fie funcția F cu F(x) = 0 dacă $x < y_m$, altfel F(x) = 1.

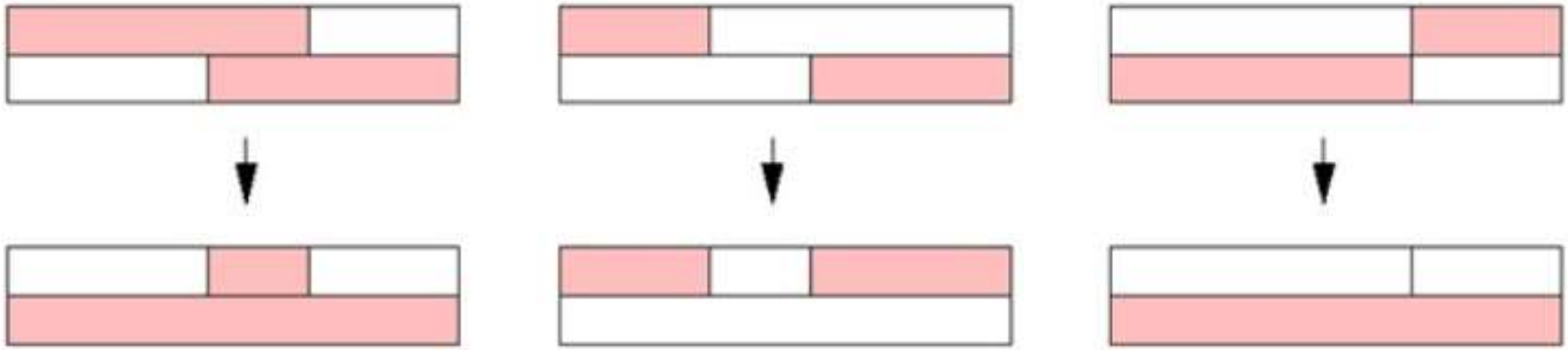
Deoarece F păstrează ordinea ≤, răspunsul algoritmului A pentru $[F(x_1), \ldots, F(x_N)]$ este $[F(y_1), \ldots, F(y_N)]$, și este de forma […, 1, 0, …] deoarece $y_m > y_{m+1}$. Contradicție.

Lema de sortare 0-1.
Comportamentul algortmului.



Lema de sortare 0-1. Comportamentul algortmului.

- Dacă avem N rânduri *mixed* atunci vom avea mai puțin de N/2 rânduri *mixed* după primul pas al sortării coloanelor.
- Fiecare pereche de rânduri *mixed* determină prin sortare cel puțin un rând pur (numai 0 sau numai 1)



Bubble sort paralel

- Ușor paralelizabilă pentru algoritmul clasic de complexitate $O(n^2)$
- Bubble sort secvențial
  - O(n) timp pentru fiecare parcurgere
  - O(n) parcurgeri
  - Putem să o paralelizăm?
- Bubble sort în paralel = Sortarea *odd-even (i.e., par-impar)*
  - Compară și schimbă perechile de indecși pari și impari
  - După n parcurgeri elementele sunt sortate
  - Putem să o paralelizăm?

Sortarea *odd-even*: Exemplu

index	1	2	3	4	5	6	7	8	
Valoare	3	2	3	8	5	6	4	1	//odd
	2	3	3	8	5	6	1	4	//even
	2	3	3	5	8	1	6	4	//odd
	2	3	3	5	1	8	4	6	//even
	...								
	1	2	3	3	4	5	6	8	//even

// secvența sortată

Sortarea *odd-even*: Algoritm

```
procedure ODD-EVEN PARALEL(n)
begin
        id := process's label
        for i := 1 to n do
        begin
                if i is odd then
                        if id is odd then
                                compare-exchange min(id + 1);
                        else
                                compare-exchange max(id - 1);
                if i is even then
                        if id is even then
                                compare-exchange min(id + 1);
                        else
                                compare-exchange max(id - 1);
        end for
        end ODD-EVEN PARALEL
```

Sortarea *odd-even*: Complexitate

- Optimizare (asemeănătoare ca la varianta secvențială)
  - Se defineste variabila partajată ***sorted*** *(i.e.,* un *flag)*, care este inițializată cu valoarea *true* la începutul fiecărei iterații (formată din cele două faze)
  - Daca un singur procesor trebuie să efectueze un *swap* atunci **sorted** := *false*.
- În interiorul buclei principale
  - Faza *odd* utilizează/necesită n/2 procesoare/comparații
  - Faza *even* utilizează necesită n/2 procesoare/comparații
- Bucla principală *for* conține *n* pași.
  - Complexitatea rulării în paralel este **O(n)**.
    - Quicksort secvențial are complexitatea O(n log n)

Algoritmul Quicksort

- Complexitatea medie este O(n log n)
- Strategia: divide-and-conquer
  - Împarte șirul în subsecvențe astfel încât fiecare element din din prima subsecvență este mai mic sau egal cu fiecare element din cea de-a doua subsecvență.
  - Pentru împărțirea secvenței se folosește un pivot.
  - Pasul combine (i.e., conquer) aplică recursiv algoritmul.
- Putem sa paralelizăm algoritmul?

Quicksort secvențial

Algoritmul are concurență intrinsecă, deci reprezintă un candidat bun pentru paralelizare:
- Listele cu valori mai mici și mai mari pot fi sortate în paralel.

{**79**, 17, 14, 65, 89, 4, 95, 22, 63, 11}

Pivot = 79

Mai mici: {17, 14, 65, 4, 22, 63, 11}

Pivot = 17

Mai mici: {14, 4, 11}

Mai mari: {64, 22, 63}

…

{4, 11, 14, 17, 22, 63, 65} este lista {17, 14, 65, 4, 22, 63, 11} sortată.

Mai mari: {89, 95}

Pivot = 89

Mai mici = Ø

Mai mari {95} => nu mai este nevoie de alt apel recursive

{89, 95} este lista {89, 95} sortată

Rezultatul final este {4, 11, 14, 17, 22, 63, 65, 79, 89, 95}

Algoritmul Quicksort secvențial

```
procedure QUICKSORT (A, q, r )// q este pozitia pivotului si r este dimensiunea lui A
begin
        if q < r then
        begin
                x := A[q]; // x este valoarea pivotului
                s := q;     // s este poziția pivotului
                for i := q + 1 to r do          // determinăm s, poziția finală a pivotului
                        if A[i] ≤ x then
                        begin
                                s := s + 1;
                                swap(A[s], A[i ]);
                        end if
                swap(A[q], A[s]);    //plasăm pivotul pe poziția finală
                QUICKSORT (A, q, s);
                QUICKSORT (A, s + 1, r );
        end if
end QUICKSORT
```

Quicksort paralel

- Presupunem că ne aflăm în cazul unui sistem cu memorie distribuită
- Fiecare proces gestionează un segment din lista nesortată
  - Lista nesortată este distribuită în mod egal între procese
- Rezultatul algoritmului:
  - Fiecare listă a fiecărui proces este sortată
  - Ultimul element lin lista procesului ***i*** este mai mic decât primul element al listei procesului ***i+1***

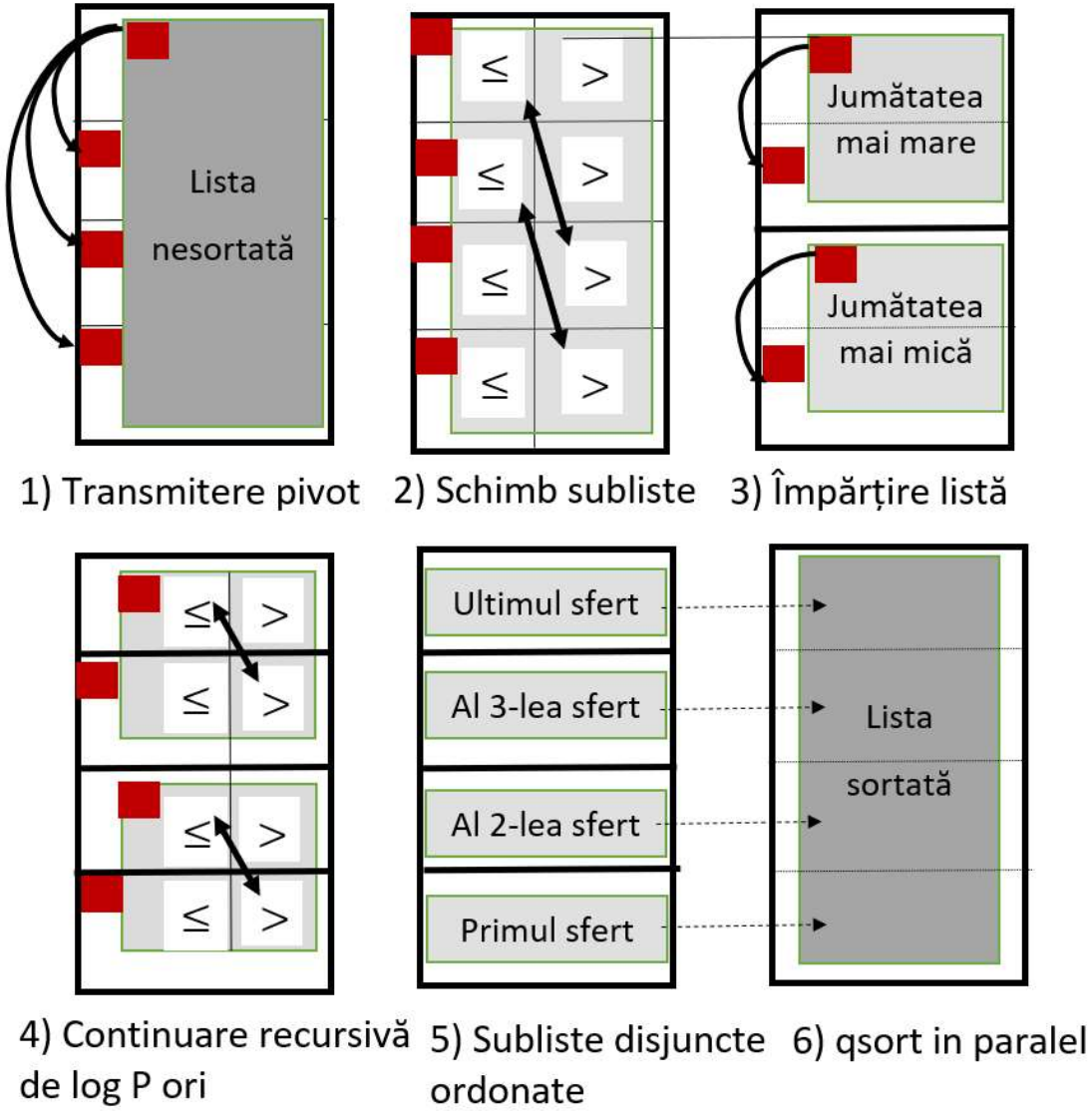
Quicksort paralel cu memorie distribuită

- Alegem aleator un pivot de la unul dintre procese și îl transmitem către toate celelalte procese (en., *broadcast*).
- Fiecare proces împarte lista sa nesortată în două subliste:
  - Cu valori mai mici sau egale cu pivotul **și** cu valori mai mari decât pivotul.
- Fiecare proces din prima jumătate a proceselor trimite sublista cu valori mici către un proces partener tot din prima jumătate a proceselor și primește la schimb sublista cu valori mai mari.
  - Astfel, sublistele cu valori mari au valori mai mari decât pivotul, iar sublistele cu valori mici au numai valori mai mici decât pivotul.
- Procesele se împart iar în două grupuri și algoritmul continuă recursiv.
- După *log P* apeluri recursive, fiecare proces deține o listă nesortată de valori complet disjuncte de valorile deținute de celelalte procese.
  - Cea mai mare valoare a procesului i este mai mică decât cea mai mică valoare a procesului i+1.
  - Fiecare proces folosește quicksort secvențial pentru a sorta lista

Exemplificare intuitivă și analiză

Balansarea încărcării este dificil de realizat

- Dacă pivotul nu este ales convenabil (i.e., valoarea mediană) atunci lista nu se va împărți în subliste de aceeași dimensiune.
- Determinarea valorii mediane are complexitate mare în sostemele distribuite.
- Soluția este să alegem ca pivot o valoare cât mai apropiată de valoarea mediană.

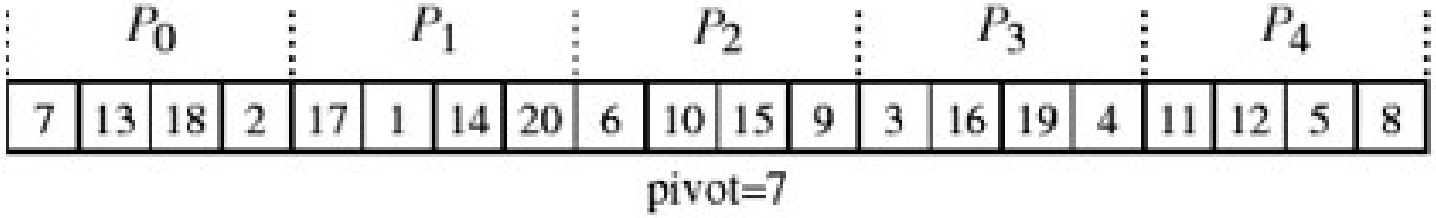


1) Transmitere pivot 2) Schimb subliste 3) Împărțire listă

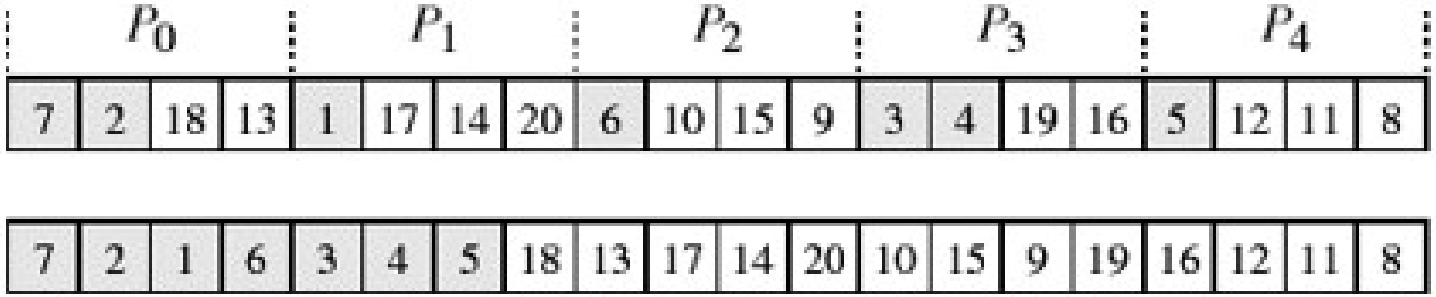
4) Continuare recursivă de log P ori 5) Subliste disjuncte ordonate 6) qsort in paralel

Quicksort paralel cu memorie partajată

Pasul 1:



Selectarea pivotului

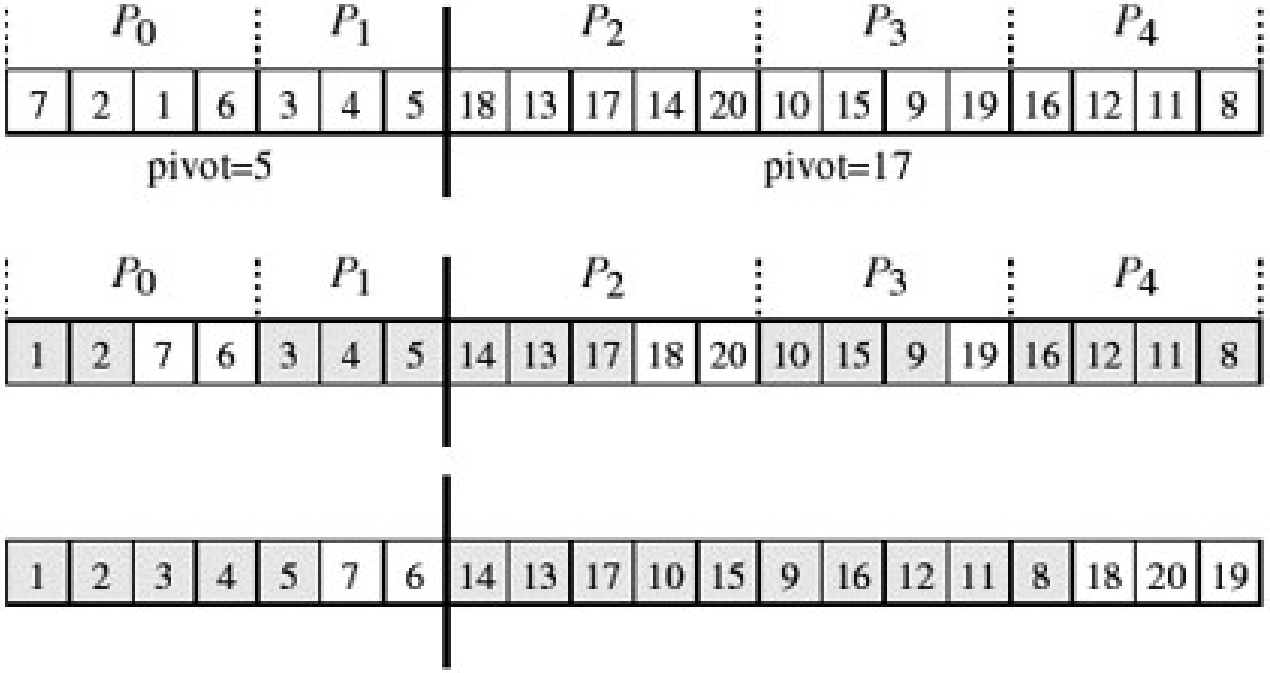


După rearanjarea locală

După rearanjarea globală

Quicksort paralel cu memorie partajată (cont.)

Pasul 2:



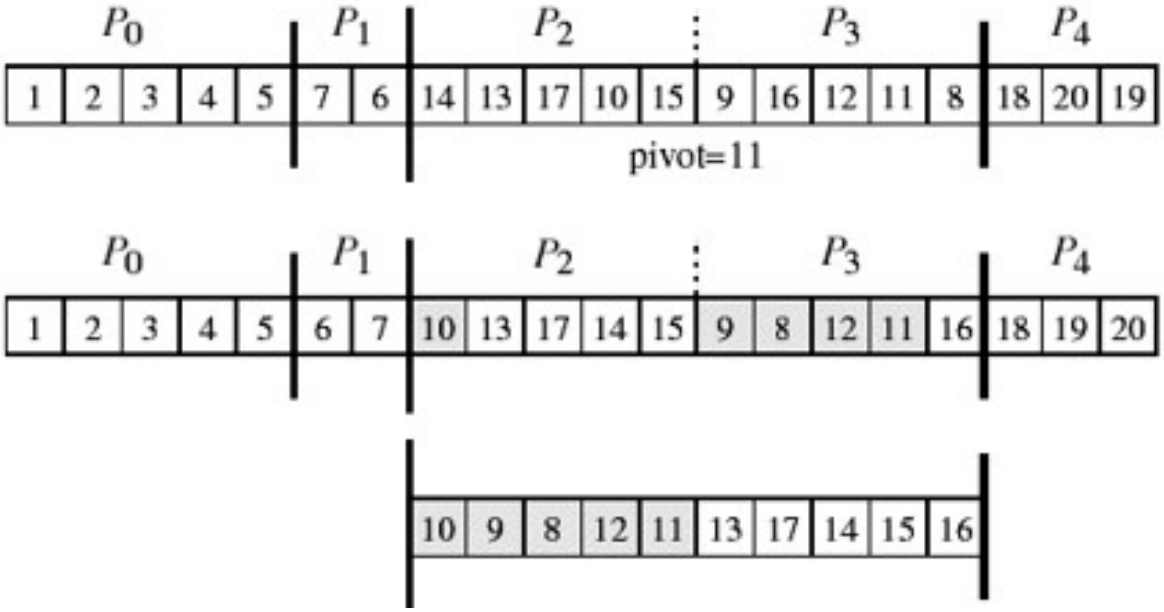
Selectarea pivotului

După rearanjarea locală

După rearanjarea globală

Quicksort paralel cu memorie partajată (cont.)

Pasul 3:



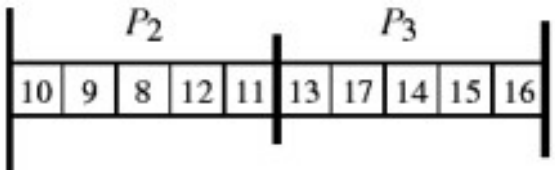
Selectarea pivotului

După rearanjarea locală

După rearanjarea globală

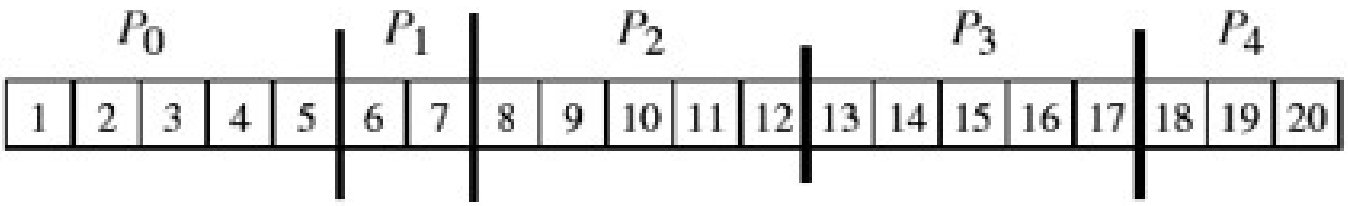
Quicksort paralel cu memorie partajată (cont.)

Pasul 4:



După rearanjarea locală

Soluția:



Sample Sort

- Algoritmul *Sample sort* este util atunci când elementele (valorile) NU sunt uniform distribuite într-un interval.
  - Creăm *m* buckets și sortăm fiecare bucket cu *quicksort*.
  - Extragem un eșantion de dimensiune *s*
  - Sortăm eșantioanele și alegem *m-1* elemente ca *splitters.*
  - Împărțim elementele în m buckets și apelăm *quicksort*
- Dacă elementele (valorile) sunt uniform distribuite într-un interval atunci putem folosi algoritmul *Bucket sort*
  - Se crează m *buckets* (= cu numărul de valori distincte) și se plasează elementele în bucket-ul corespunzător
  - Complexitatea este O(n log (n/m))
  - Dacă m=n, folosim valoarea ca index pentru a obține timp O(n)

Algoritmul *Sample Sort* paralel

Pas 1. Se împarte lista inițială nesortată în *P* segmente egale. Fiecare segment se sortează cu ajutorul quicksort. Din fiecare segment se selectează elementele care se află la pozițiile 0, n/P^2, $2n/P^2$, ..., $(P-1)n/P^2$

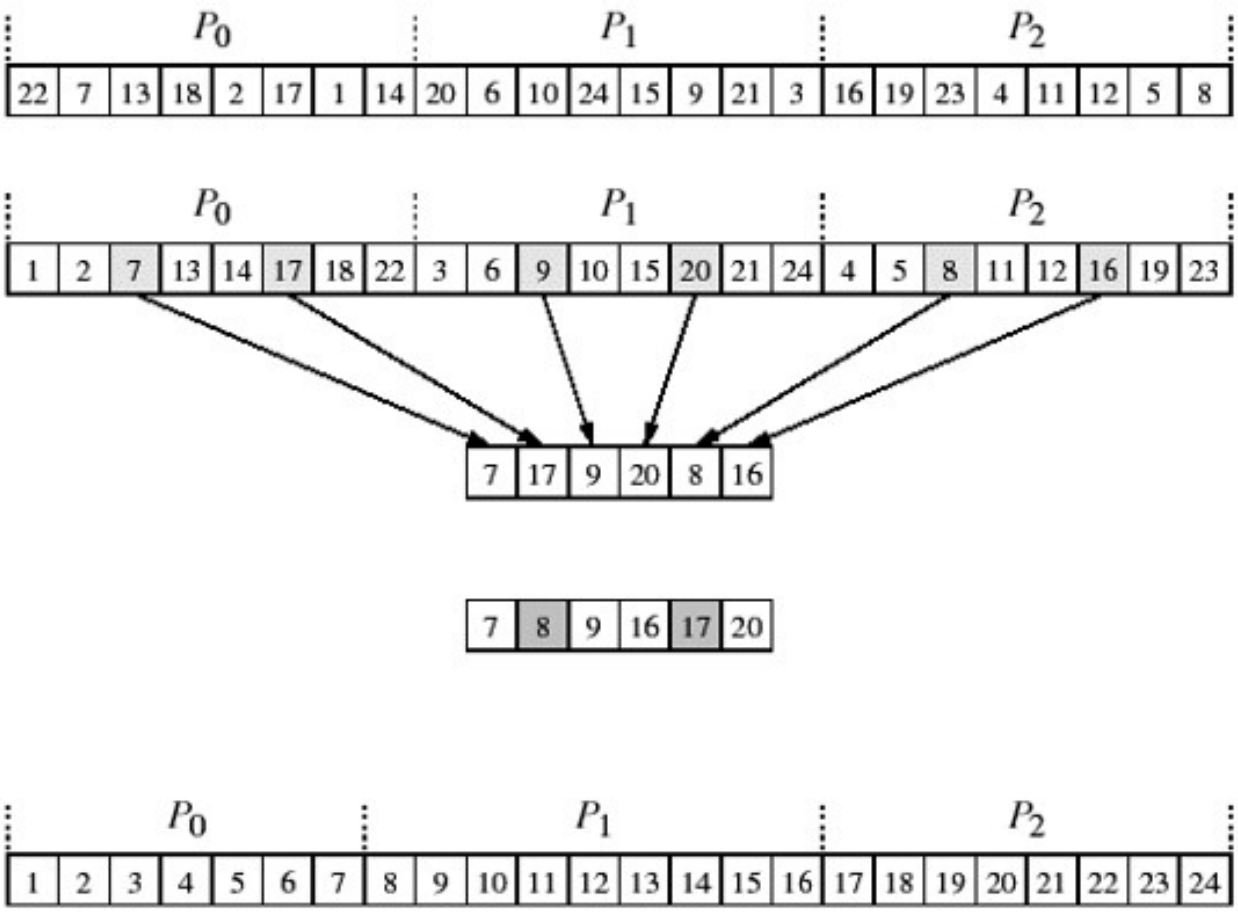
Aceste valori reprezintă eșantionul reprezentativ al segmentelor sortate local.

Pas 2. Se soretază eșantionul obținut la pasul anterior. La acest pas selectăm P-1 valori pivot din eșantionul reprezentativ. Cele P-1 valori sunt transmise către toate procesele/segmentele. Fiecare proces împarte lista sa sortată în P subliste disjuncte corespunzătoare valorilor pivot primite.

Pas 3. Fiecare proces *i* păstrează partiția sa *i* și transmite partiția j procesului j pentru toate valorile j diferite de i.

Pas 4. Fiecare proces combină cele P partiții pentru a obține lista ordonată.

Sample Sort paralel: exemplu



Valorile inițiale

Sortare locală (qs) și Selectarea eșantionului

Sortarea eșantionului

Selectarea spliter-ului global

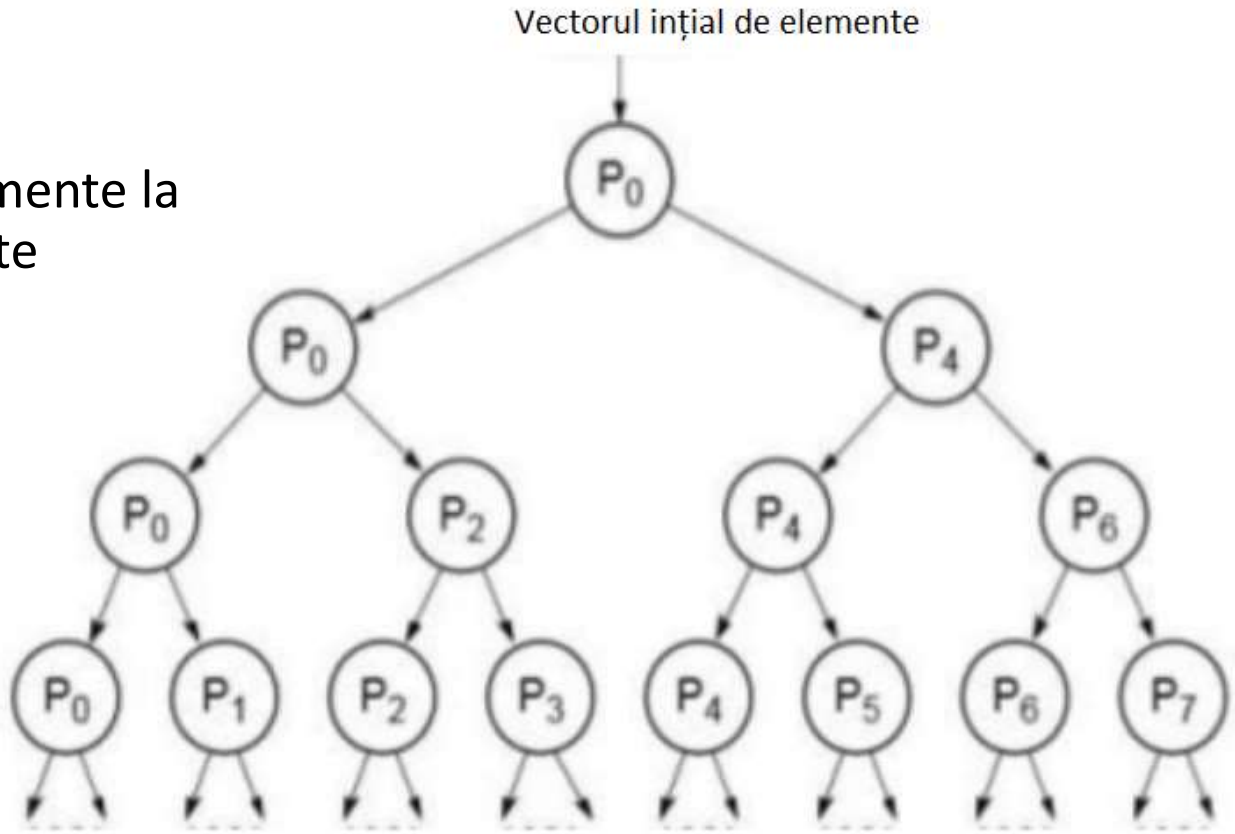
Atribuirea finală

Merge Sort

- Strategia este divide-and-conquer
  - Împarte (en., divide) vectorul de elemente în două părți
  - Aplică aceeași strategie pentru fiecare parte definită anterior
  - Atunci când două părți ajung sa fie ordonate ele trebuie unite (en., merge, conquer) într-un vector ordonat care contine elementele celor două părți.
  - Complexitatea medie **O(n log n)**
    - $T(n) = \begin{cases} b & \text{pentru } n=1 \\ 2T(n/2) + b*n & \text{pentru } n>1 \end{cases}$
- Merge Sort în paralel: Definim două task-uri:
  - Divide și
  - Conquer

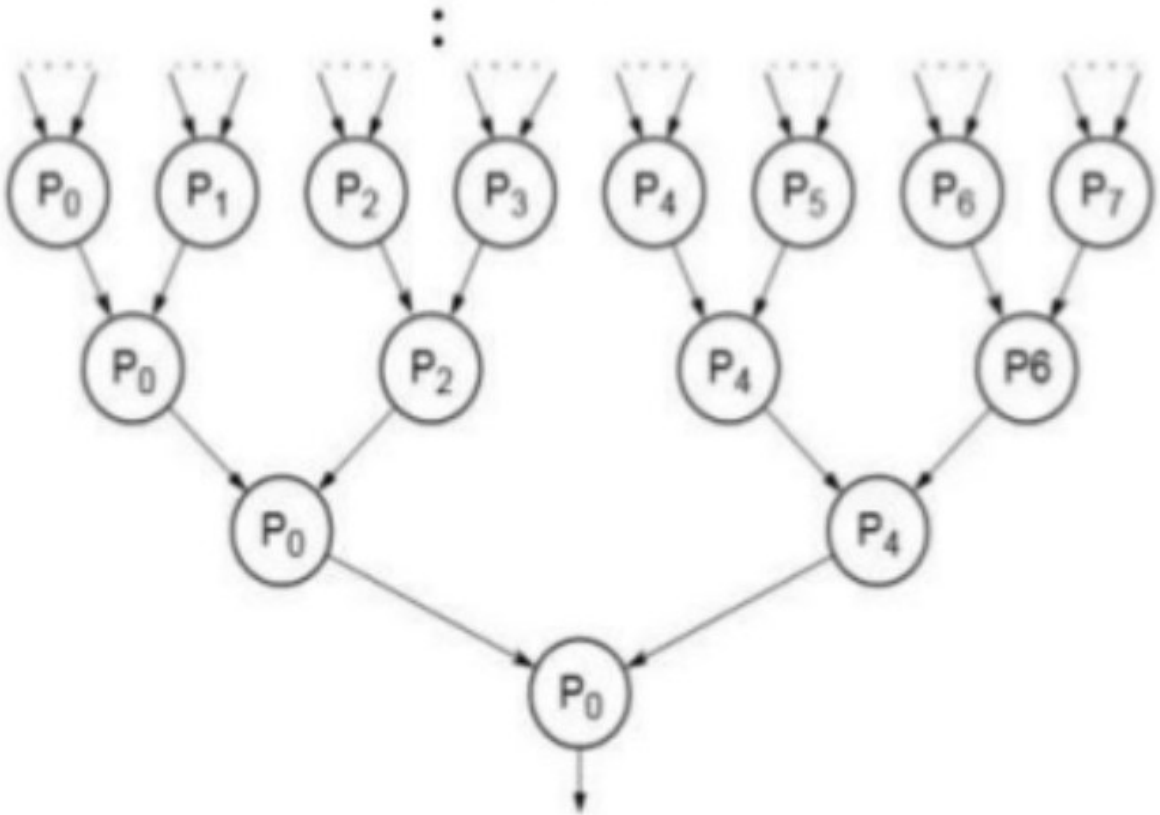
Merge Sort (cont.)

- Divide
  - Împarte vectorul de elemente la două procesoare distincte
  - Paralelismul este limitat aproape de rădăcină.



Merge Sort (cont.)

- Conquer
  - Input: doi vectori sortați
  - Output: un vector sortat care conține toate elementele celor doi vectori din input.



Algoritmul Merge Sort paralel

```
Algoritm: MergeSort (A)
if (|A| = 1)  then return A
else
 #Do in parallel
        L := MergeSort (A[0 … |A|/2])
        R := MergeSort (|A|/2] … |A|)
  return merge (L, R)
```

Algoritmul Merge Sort parallel: Exemplul 1

Input: secvența 4, 3,2, 1 și avem la dispoziție 2 procesoare P0 și P1

P0 [4, 3, 2, 1]

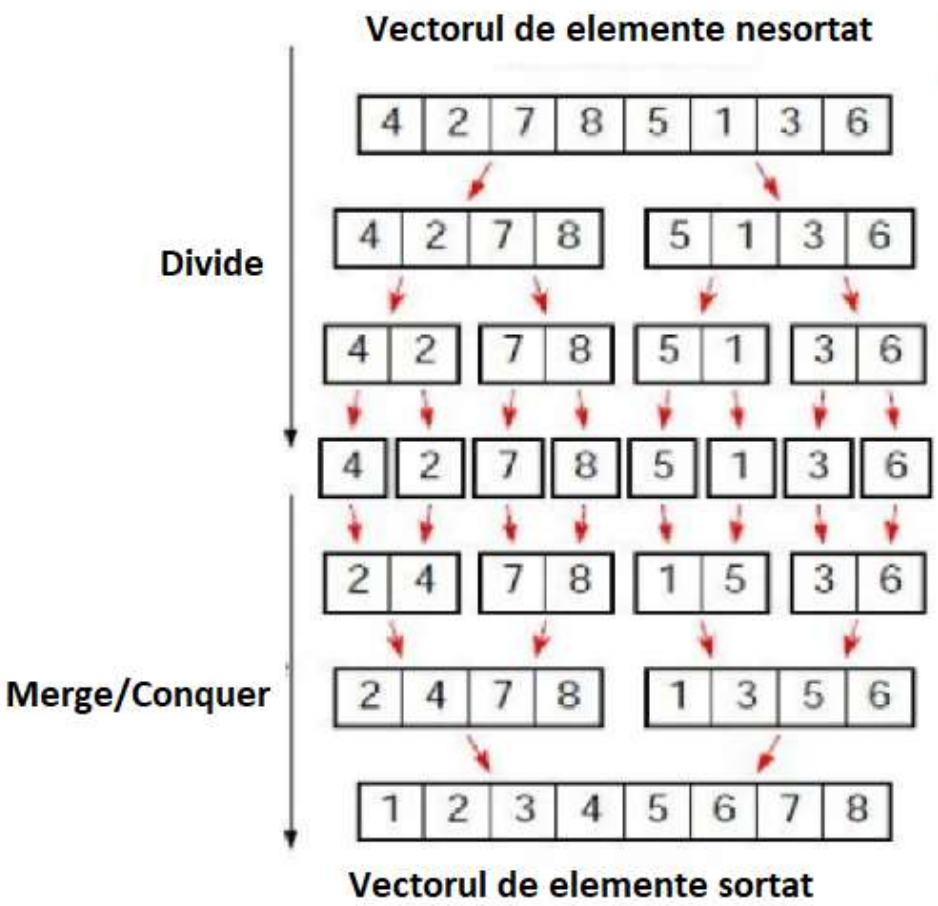
P0 [4, 3] P1 [2, 1]

[4] [3] [2] [1]

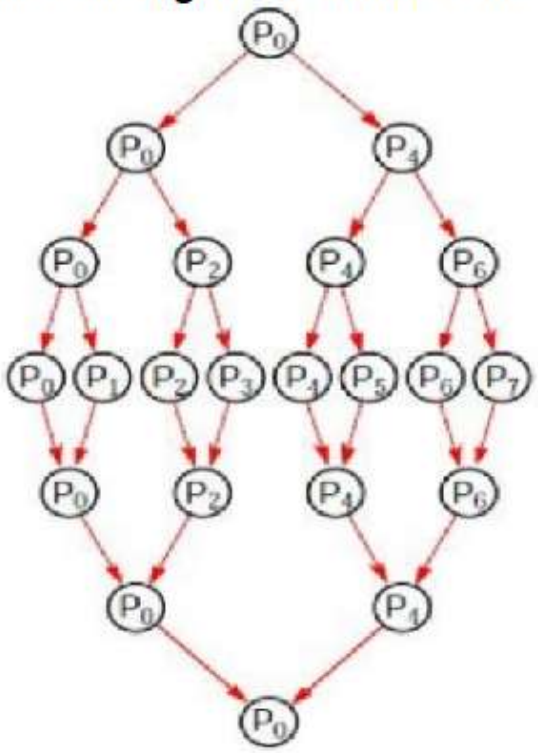
P0 [3, 4] P1 [1, 2]

P0 [1, 2, 3, 4]

Algoritmul Merge Sort parallel: Exemplul 2



Task-urile sunt alocate celor 9 procesoare care sunt organizate arborescent.



Algoritmul Merge Sort parallel (cont.)

Idea principală: Paralelizarea procesări subproblemelor

Paralelizarea maximă se obține atunci când avem câte un procesor pentru fiecare nod la fiecare nivel din arbore.

Complexitatea:

- Algoritmul merge sort secvențial, O(n log n)
- În paralel, cu *n* procesoare
  - log n pentru divide
  - log n pentru merge
  - log n + log n = 2 log n
  - O(log n)

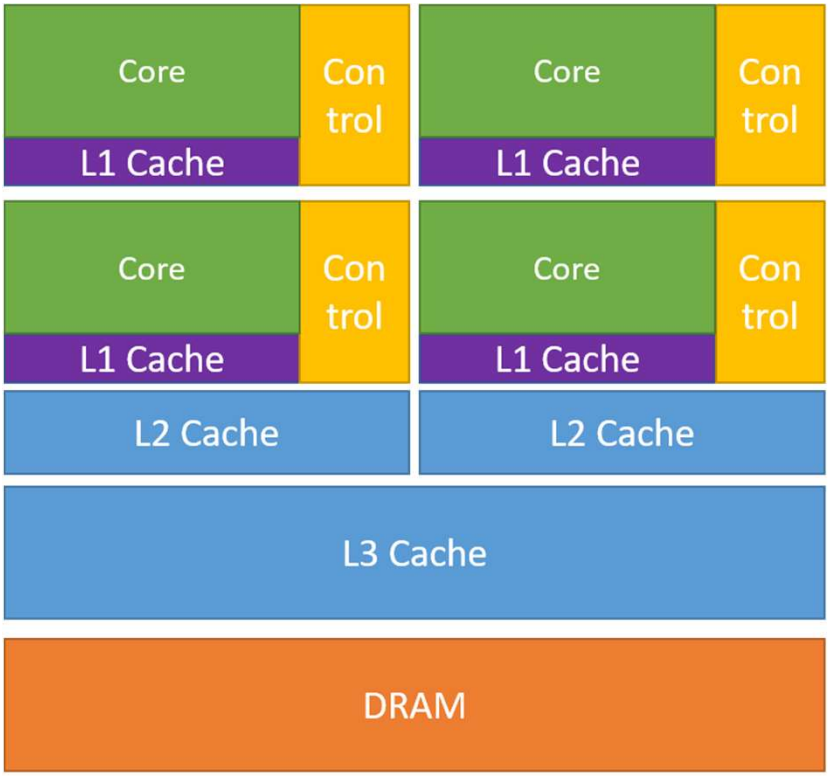
CUDA: Scurtă prezentare

- CUDA (Compute Unified Device Architecture) este o platformă de calcul paralel care pune la dispoziție API creat de Nvidia pentru dezvoltarea de aplicații care utilizează GPU (graphics processing unit).
- Platforma CUDA reprezintă un nivel software care permite accesul și folosirea directă a GPU-ului.
- Limbaje de programare folosite: C, C++, Fortran.
  - Se permite dezvoltarea mai ușoară de aplicații paralele comparativ cu Direct3D și OpenGL folosite în procesarea grafică.
- CUDA-powered GPUs supportă framework-uri ca OpenACC sau Open CL

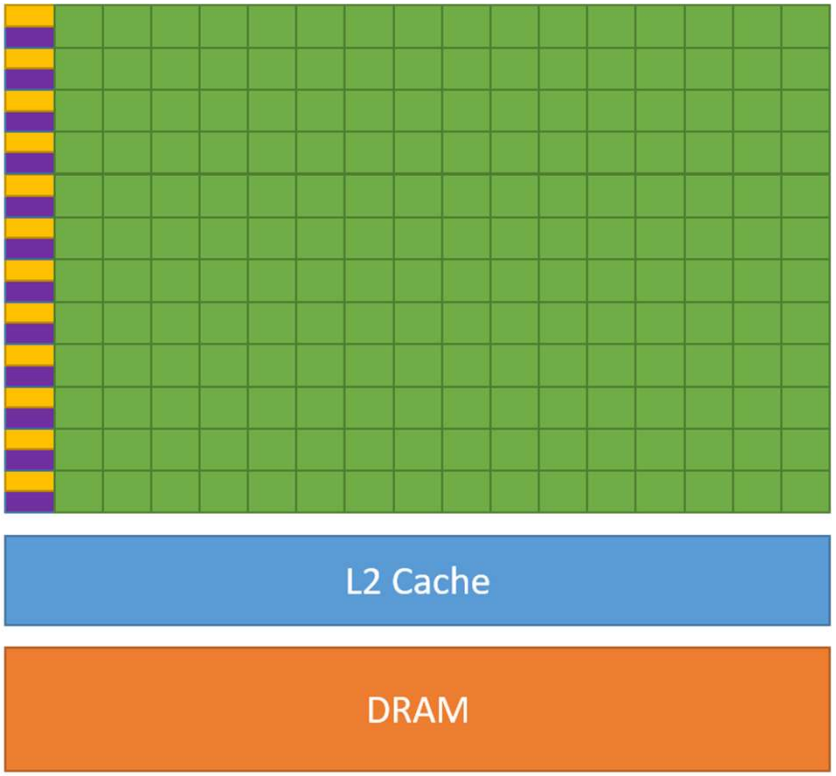
CUDA: Beneficiile utilizării GPU

- GPU are un debit (en., throughput) mult mai mare de instrucțiuni executate și o lățime de bandă mult mai mare decât CPU pentru costuri și putere consumată similare.
- Capabilitățile eferite de GPU comparativ cu CPU sunt datorate scopurilor diferite pe care le au în vedere proiectanții.
  - CPU: scopul principal este să execute o secvență de operații într-un fir de execuție.
  - GPU: scopul principal este să execute mii de secvențe de operații în paralel. Performața unui singur fir de execuție este mai mică, dar debitul (i.e., cantitatea de date procesată) este mult mai mare.
  - GPU este specializat pentru executarea de multe operațiuni în paralel astfel încât cei mai mulți tranzistori sunt dedicați procesarii de date și NU gestionării de cache-uri sau controlul fluxului aplicației.

CPU vs. GPU



CPU



GPU

CPU vs. GPU

- GPU:
  - Defică mult mai mulți tranzistori pentru procesare (i.e., operații FLP) ceea ce ajută mult procesările paralele.
  - Are latență mică la efectuarea calculelor deoarece nu se bazează pe cache-uri si nici pe fluxuri complicate de calcul.
- Aplicațiile sunt de obicei un mix între părți secvențiale și paralele
  - Părțile secvențiale sunt executate de CPU
  - Părțile paralele sunt executate de GPU
  - Astfel încât de maximizează performanța generală.

Aplicații care utilizează GPU

GPU Computing Applications

Libraries and Middleware

cuDNN TensorRT	cuFFT cuBLAS cuRAND cuSPARSE	CULA MAGMA	Thrust NPP	VSIPL SVM OpenCurrent	PhysX OptiX iRay	MATLAB Mathematica

Programming Languages

C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)

CUDA-Enabled NVIDIA GPUs

NVIDIA Ampere Architecture (compute capabilities 8.x)				Tesla A Series
NVIDIA Turing Architecture (compute capabilities 7.x)		GeForce 2000 Series	Quadro RTX Series	Tesla T Series
NVIDIA Volta Architecture (compute capabilities 7.x)	DRIVE/JETSON AGX Xavier		Quadro GV Series	Tesla V Series
NVIDIA Pascal Architecture (compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series	Tesla P Series
	Embedded	Consumer Desktop/Laptop	Professional Workstation	Data Center

Model de programare paralel CUDA

- Provocarea este să ofere o curbă de învățare rapidă pentru programatorii C/C++.
- Ingrediente fundamentale
  - Firele de execuție
  - Memoria partajată
  - Sincronizarea barierelor
- Aceste ingredient oferă paralelism cu granularitate mică a datelor și paralelismul firelor, alături de paralelismul de granularitate mare și paralelismu task-urilor.

Model de programare paralel CUDA

Kernels reprezintă în CUDA ceea ce reprezintă funcțiile din C++.

- Atunci când sunt apelate ele se rulează de N ori în paralel pe N fire diferite.
- Este definit de declarația __global__ și de numărul de fire CUDA care execută kernel-ul.
- <<<...>>> este sintaxa pentru configurarea execuției.

```
// Kernel definition
__global__ void VecAdd(float* A, float* B,
float* C)
{
  int i = threadIdx.x;
  C[i] = A[i] + B[i];
}

int main()
{ ...
// each of the N threads that execute
//VecAdd() performs one pair-wise addition.
VecAdd<<<1, N>>>(A, B, C);
}
```

Model de programare paralel CUDA (cont.)

Ierarhia de fire

threadIdx este un vector cu 3 componente. Firele pot fi identificate folosind indecși 1D, 2D sau 3D care formează astfel blocuri de fire (en., thread block). Acest mod de lucru perimite lucrul natural cu date de tip vector, matrice sau volum.

```
// Kernel definition __global__
void MatAdd(float A[N][N], float B[N][N],
float C[N][N])
{
  int i = threadIdx.x;
  int j = threadIdx.y;
  C[i][j] = A[i][j] + B[i][j];
}

int main()
{ ... // Kernel invocation with one block
       //of N * N * 1 threads
  int numBlocks = 1;
  dim3 threadsPerBlock(N, N);
  MatAdd<<<numBlocks, threadsPerBlock>>>(A, B, C);
...
}
```

Model de programare paralel CUDA (cont.)

- Ierarhia memoriei
  - Firele CUDA pot accesa date din maimulte zone de memorie în timpul execuției.
  - Fiecare fir are o zona de memorie locală privată.
  - Fiecare bloc de fire (en., thread block) are o zonă de memorie partajată care este vizibilă/accesibilă tuturor firelor din același bloc.
  - Toate firele au acces la memoria globală.
- Programarea eterogenă (secvențială și paralelă)
  - Kernel-urile se execută pe GPU
  - Codul C++ (secvențial) se execută pe CPU.

CUDA: Dezvoltarea aplicațiilor

- Compilarea cu nvcc;// pentru codul care conține *C++ Language Extensions // (i.e., __global__, __device__, etc.)*
  - *Compilarea offline și just-in-time*
- Compatibilitatea binară
- Compatibilitatea PTX
- Compatibilitatea aplicației
- CUDA Runtime
  - Inițializare
  - Memoria și managementul memoriei
  - Memoria partajată
  - Etc.

CUDA: Referințe

https://docs.nvidia.com/cuda/cuda-c-programming-guide/

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#introduction

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#programming-model

https://docs.nvidia.com/cuda/cuda-c-programming-guide/#programming-interface